Peering 피어링

개요

Peering 모듈은 IPFS 노드가 특정 피어들과의 연결을 지속적으로 유지하는 기능을 제공합니다. 이 모듈은 지정된 피어들과의 연결이 끊어졌을 때 자동으로 재연결을 시도하며, 백오프(back-off) 알고리즘을 사용하여 네트워크 부하를 최소화합니다. 피어링은 네트워크 안정성을 향상시키고 중요한 피어들과의 연결을 보장하는 데 사용됩니다.

주요 개념

피어링 서비스

피어링 서비스는 다음과 같은 기능을 제공합니다:

- **자동 재연결**: 연결이 끊어진 피어에 대해 자동으로 재연결 시도
- **백오프 알고리즘**: 연결 실패 시 점진적으로 재시도 간격을 늘려 네트워크 부하 감소
- **연결 보호**: 연결 관리자가 중요한 피어 연결을 임의로 끊지 않도록 보호
- **동적 관리**: 실행 중에도 피어 추가/제거 가능

상태 관리

피어링 서비스는 세 가지 상태를 가집니다:

- **StateInit**: 초기 상태, 아직 시작되지 않음
- **StateRunning**: 실행 중, 피어 연결을 적극적으로 관리
- **StateStopped**: 중지됨, 더 이상 연결 관리하지 않음

백오프 메커니즘

- **초기 지연**: 5초로 시작
- **최대 백오프**: 10분까지 증가
- **지터**: 최대 백오프의 10% 범위에서 랜덤화하여 동시 연결 시도 방지

기본 사용법

기본 피어링 서비스 생성

```
package main

import (
"context"
"log"
"time"
"github.com/ipfs/boxo/peering"
"github.com/libp2p/go-libp2p"
"github.com/libp2p/go-libp2p/core/peer"
"github.com/multiformats/go-multiaddr"
)

func main() {
// libp2p 호스트 생성
host, err := libp2p.New()
if err != nil {
```

```
log.Fatal(err)
}
defer host.Close()
// 피어링 서비스 생성
ps := peering.NewPeeringService(host)
// 피어링할 피어 정보 정의
peerInfo := peer.AddrInfo{
ID: "12D3KooWGRPLCuizAiJWLtktrdxKKoKG5b3BtG4pEKbpzTLxmdSZ",
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/104.131.131.82/tcp/4001"),
},
}
// 피어 추가
ps.AddPeer(peerInfo)
// 피어링 서비스 시작
if err := ps.Start(); err != nil {
log.Fatal(err)
}
defer ps.Stop()
// 서비스 상태 확인
log.Printf("피어링 서비스 상태: %s", ps.GetState())
// 피어링된 피어 목록 확인
peers := ps.ListPeers()
```

```
log.Printf("피어링된 피어 수: %d", len(peers))
// 애플리케이션 로직...
time.Sleep(60 * time.Second)
}
```

동적 피어 관리

```
func dynamicPeerManagement() error {
host, err := libp2p.New()
if err != nil {
return err
}
defer host.Close()
ps := peering.NewPeeringService(host)
// 서비스 시작
if err := ps.Start(); err != nil {
return err
}
defer ps.Stop()
// 여러 피어 추가
peers := []peer.AddrInfo{
{
ID: "12D3KooWGRPLCuizAiJWLtktrdxKKoKG5b3BtG4pEKbpzTLxmdSZ",
```

```
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/104.131.131.82/tcp/4001"),
},
},
{
ID: "12D3KooWHBFS8HVjRuQAJAqSjsKm8yAaEyYQp8HoEJNpMa7bHgvr",
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/104.131.131.83/tcp/4001"),
},
},
}
for _, peerInfo := range peers {
ps.AddPeer(peerInfo)
log.Printf("피어 추가됨: %s", peerInfo.ID)
}
// 잠시 대기
time.Sleep(30 * time.Second)
// 첫 번째 피어 제거
ps.RemovePeer(peers[0].ID)
log.Printf("피어 제거됨: %s", peers[0].ID)
// 현재 피어링된 피어 목록 확인
currentPeers := ps.ListPeers()
log.Printf("현재 피어링된 피어 수: %d", len(currentPeers))
```

```
return  nil
}
```

고급 활용

피어링 매니저 구현

```
package main

import (
"context"
"encoding/json"
"fmt"
"log"
"os"
"path/filepath"
"sync"
"time"
"github.com/ipfs/boxo/peering"
"github.com/libp2p/go-libp2p"
"github.com/libp2p/go-libp2p/core/host"
"github.com/libp2p/go-libp2p/core/network"
"github.com/libp2p/go-libp2p/core/peer"
```

```
)

type PeeringManager struct {
host host.Host
service *peering.PeeringService
configFile  string
mu sync.RWMutex
peers  map[peer.ID]peer.AddrInfo
// 모니터링
connectionEvents  chan ConnectionEvent
stopMonitoring  chan  struct{}
}

type ConnectionEvent struct {
PeerID peer.ID
Connected  bool
Timestamp time.Time
}

type PeeringConfig struct {
Peers []peer.AddrInfo `json:"peers"`
}
```

```go
func NewPeeringManager(host host.Host, configFile string) *PeeringManager {
return &PeeringManager{
host: host,
service: peering.NewPeeringService(host),
configFile: configFile,
peers: make(map[peer.ID]peer.AddrInfo),
connectionEvents: make(chan ConnectionEvent, 100),
stopMonitoring: make(chan struct{}),
}
}

func (pm *PeeringManager) Start() error {
// 설정 파일에서 피어 로드
if err := pm.loadConfig(); err != nil {
log.Printf("설정 로드 실패: %v", err)
}
// 로드된 피어들을 서비스에 추가
pm.mu.RLock()
for _, peerInfo := range pm.peers {
pm.service.AddPeer(peerInfo)
}
pm.mu.RUnlock()
// 피어링 서비스 시작
```

```
if err := pm.service.Start(); err != nil {
return fmt.Errorf("피어링 서비스 시작 실패: %w", err)
}
// 연결 모니터링 시작
go pm.monitorConnections()
log.Printf("피어링 매니저 시작됨 (피어 수: %d)", len(pm.peers))
return nil
}


func (pm *PeeringManager) Stop() error {
// 모니터링 중지
close(pm.stopMonitoring)
// 피어링 서비스 중지
pm.service.Stop()
// 설정 저장
if err := pm.saveConfig(); err != nil {
log.Printf("설정 저장 실패: %v", err)
}
log.Printf("피어링 매니저 중지됨")
return nil
}


func (pm *PeeringManager) AddPeer(peerInfo peer.AddrInfo) {
```

```
pm.mu.Lock()
pm.peers[peerInfo.ID] = peerInfo
pm.mu.Unlock()
pm.service.AddPeer(peerInfo)
log.Printf("피어 추가됨: %s", peerInfo.ID)
}

func (pm *PeeringManager) RemovePeer(peerID peer.ID) {
pm.mu.Lock()
delete(pm.peers, peerID)
pm.mu.Unlock()
pm.service.RemovePeer(peerID)
log.Printf("피어 제거됨: %s", peerID)
}

func (pm *PeeringManager) ListPeers() []peer.AddrInfo {
pm.mu.RLock()
defer pm.mu.RUnlock()
peers := make([]peer.AddrInfo, 0, len(pm.peers))
for _, peerInfo := range pm.peers {
peers = append(peers, peerInfo)
}
return peers
```

```
}

func (pm *PeeringManager) GetConnectionStatus() map[peer.ID]bool {
pm.mu.RLock()
defer pm.mu.RUnlock()
status := make(map[peer.ID]bool)
for peerID := range pm.peers {
status[peerID] = pm.host.Network().Connectedness(peerID) == network.Connected
}
return status
}

func (pm *PeeringManager) loadConfig() error {
data, err := os.ReadFile(pm.configFile)
if err != nil {
if os.IsNotExist(err) {
return nil // 설정 파일이 없으면 무시
}
return err
}
var config PeeringConfig
if err := json.Unmarshal(data, &config); err != nil {
return err
```

```
}
pm.mu.Lock()
defer pm.mu.Unlock()
for _, peerInfo := range config.Peers {
pm.peers[peerInfo.ID] = peerInfo
}
return nil
}

func (pm *PeeringManager) saveConfig() error {
pm.mu.RLock()
peers := make([]peer.AddrInfo, 0, len(pm.peers))
for _, peerInfo := range pm.peers {
peers = append(peers, peerInfo)
}
pm.mu.RUnlock()
config := PeeringConfig{Peers: peers}
data, err := json.MarshalIndent(config, "", " ")
if err != nil {
return err
}
// 디렉토리 생성
if err := os.MkdirAll(filepath.Dir(pm.configFile), 0755); err != nil {
```

```
return err
}
return os.WriteFile(pm.configFile, data, 0644)
}


func (pm *PeeringManager) monitorConnections() {
ticker := time.NewTicker(10 * time.Second)
defer ticker.Stop()
// 이전 연결 상태 추적
prevStatus := make(map[peer.ID]bool)
for {
select {
case <-ticker.C:
currentStatus := pm.GetConnectionStatus()
// 연결 상태 변화 감지
for peerID, connected := range currentStatus {
if prevConnected, exists := prevStatus[peerID]; !exists || prevConnected != con
event := ConnectionEvent{
PeerID: peerID,
Connected: connected,
Timestamp: time.Now(),
}
select {
```

```
case  pm.connectionEvents <- event:
default:
// 채널이 가득 찬 경우 이벤트 드롭
}
if  connected {
log.Printf("피어 연결됨: %s", peerID)
} else {
log.Printf("피어 연결 끊어짐: %s", peerID)
}
}
}
prevStatus = currentStatus
case <-pm.stopMonitoring:
return
}
}
}

func (pm *PeeringManager) GetConnectionEvents() <-chan ConnectionEvent {
return  pm.connectionEvents
}

func  main() {
```

```
// libp2p 호스트 생성
host, err := libp2p.New()
if err != nil {
log.Fatal(err)
}
defer host.Close()
// 피어링 매니저 생성
configFile := filepath.Join(os.TempDir(), "peering_config.json")
manager := NewPeeringManager(host, configFile)
// 매니저 시작
if err := manager.Start(); err != nil {
log.Fatal(err)
}
defer manager.Stop()
// 테스트 피어 추가
testPeer := peer.AddrInfo{
ID: "12D3KooWGRPLCuizAiJWLtktrdxKKoKG5b3BtG4pEKbpzTLxmdSZ",
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/104.131.131.82/tcp/4001"),
},
}
manager.AddPeer(testPeer)
// 연결 이벤트 모니터링
```

```
go func() {
for event := range manager.GetConnectionEvents() {
log.Printf("연결 이벤트: 피어=%s, 연결됨=%v, 시간=%s",
event.PeerID, event.Connected, event.Timestamp.Format(time.RFC3339))
}
}()
// 주기적으로 상태 출력
ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()
for i := 0; i < 6; i++ {
<-ticker.C
peers := manager.ListPeers()
status := manager.GetConnectionStatus()
log.Printf("=== 피어링 상태 ===")
log.Printf("총 피어 수: %d", len(peers))
connectedCount := 0
for peerID, connected := range status {
if connected {
connectedCount++
}
log.Printf(" %s: %v", peerID, connected)
}
log.Printf("연결된 피어 수: %d", connectedCount)
```

```
}

}
```

설정 옵션

백오프 설정

피어링 모듈의 백오프 설정은 내부적으로 관리되지만, 다음과 같은 상수들을 이해하는 것이 도움됩니다:

```
const (

// maxBackoff: 재연결 시도 간 최대 시간

maxBackoff = 10 * time.Minute

// maxBackoffJitter: 최대 백오프의 지터 비율 (10%)

maxBackoffJitter = 10

// ConnmgrTag: 연결 관리자에서 사용하는 태그

ConnmgrTag = "ipfs-peering"

// initialDelay: 초기 재연결 지연 시간

initialDelay = 5 * time.Second

)
```

연결 관리자 통합

피어링 서비스는 libp2p의 연결 관리자와 통합되어 중요한 연결을 보호합니다:

```
// 피어 추가 시 자동으로 연결 보호
ps.AddPeer(peerInfo) // 내부적으로 ConnManager().Protect() 호출


// 피어 제거 시 보호 해제
ps.RemovePeer(peerID) // 내부적으로 ConnManager().Unprotect() 호출
```

실제 예제

서버 환경에서의 피어링

```
package main


import (
"context"
"log"
"os"
"os/signal"
"syscall"
"time"
"github.com/ipfs/boxo/peering"
"github.com/libp2p/go-libp2p"
"github.com/libp2p/go-libp2p/core/peer"
"github.com/libp2p/go-libp2p/p2p/net/connmgr"
```

```
)

func main() {
// 연결 관리자 생성 (최소 10개, 최대 100개 연결)
cm, err := connmgr.NewConnManager(10, 100, connmgr.WithGracePeriod(time.Minute))
if err != nil {
log.Fatal(err)
}
// libp2p 호스트 생성
host, err := libp2p.New(
libp2p.ListenAddrStrings("/ip4/0.0.0.0/tcp/4001"),
libp2p.ConnectionManager(cm),
)
if err != nil {
log.Fatal(err)
}
defer host.Close()
log.Printf("노드 시작됨: %s", host.ID())
log.Printf("수신 주소: %v", host.Addrs())
// 피어링 서비스 생성
ps := peering.NewPeeringService(host)
// 중요한 피어들 추가 (예: 다른 서버 노드들)
importantPeers := []peer.AddrInfo{
```

```
// 실제 환경에서는 설정 파일에서 로드
{
ID: "12D3KooWGRPLCuizAiJWLtktrdxKKoKG5b3BtG4pEKbpzTLxmdSZ",
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/10.0.1.100/tcp/4001"),
},
},
{
ID: "12D3KooWHBFS8HVjRuQAJAqSjsKm8yAaEyYQp8HoEJNpMa7bHgvr",
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/10.0.1.101/tcp/4001"),
},
},
}
for _, peerInfo := range importantPeers {
ps.AddPeer(peerInfo)
log.Printf("중요 피어 추가됨: %s", peerInfo.ID)
}
// 피어링 서비스 시작
if err := ps.Start(); err != nil {
log.Fatal(err)
}
// 상태 모니터링 고루틴
```

```
go func() {
ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()
for {
select {
case <-ticker.C:
peers := ps.ListPeers()
connectedCount := 0
for _, peerInfo := range peers {
if host.Network().Connectedness(peerInfo.ID) == network.Connected {
connectedCount++
}
}
log.Printf("피어링 상태: %d/%d 연결됨", connectedCount, len(peers))
log.Printf("전체 연결 수: %d", len(host.Network().Peers()))
}
}
}()
// 우아한 종료 처리
sigCh := make(chan os.Signal, 1)
signal.Notify(sigCh, syscall.SIGINT, syscall.SIGTERM)
log.Printf("서버 실행 중... (종료하려면 Ctrl+C)")
<-sigCh
```

```
log.Printf("종료 신호 수신, 피어링 서비스 중지 중...")
ps.Stop()
log.Printf("서버 종료됨")
}
```

클라이언트 환경에서의 피어링

```
func clientPeering() error {
// 클라이언트용 가벼운 설정
host, err := libp2p.New(
libp2p.ListenAddrStrings("/ip4/127.0.0.1/tcp/0"), // 로컬만 수신
libp2p.DisableRelay(), // 릴레이 비활성화
)
if err != nil {
return err
}
defer host.Close()
ps := peering.NewPeeringService(host)
// 신뢰할 수 있는 게이트웨이나 부트스트랩 노드와 피어링
gatewayPeer := peer.AddrInfo{
ID: "12D3KooWGRPLCuizAiJWLtktrdxKKoKG5b3BtG4pEKbpzTLxmdSZ",
Addrs: []multiaddr.Multiaddr{
multiaddr.StringCast("/ip4/104.131.131.82/tcp/4001"),
```

```
},
}
ps.AddPeer(gatewayPeer)
if err := ps.Start(); err != nil {
return err
}
defer ps.Stop()
// 클라이언트 로직...
time.Sleep(60 * time.Second)
return nil
}
```

모범 사례

1. 적절한 피어 선택

```
// 좋은 피어링 대상:
// - 안정적인 서버 노드들
// - 지리적으로 가까운 노드들
// - 신뢰할 수 있는 조직의 노드들

// 피하는 것이 좋은 피어링 대상:
// - 불안정한 클라이언트 노드들
```

```
// - 네트워크 지연이 큰 노드들
// - 알 수 없는 노드들
```

2. 피어 수 관리

```
// 서버 환경: 5-20개 피어
importantPeers := loadServerPeers() // 5-20개

// 클라이언트 환경: 1-5개 피어
importantPeers := loadClientPeers() // 1-5개

// 너무 많은 피어링은 리소스 낭비
// 너무 적은 피어링은 안정성 저하
```

3. 모니터링과 알림

```
func monitorPeeringHealth(ps *peering.PeeringService, host host.Host) {
ticker := time.NewTicker(5 * time.Minute)
defer ticker.Stop()
for {
<-ticker.C
peers := ps.ListPeers()
connectedCount := 0
```

```
for _, peerInfo := range peers {
if host.Network().Connectedness(peerInfo.ID) == network.Connected {
connectedCount++
}
}
healthRatio := float64(connectedCount) / float64(len(peers))
if healthRatio < 0.5 {
log.Printf("경고: 피어링 건강도 낮음 (%d/%d = %.1f%%)",
connectedCount, len(peers), healthRatio*100)
// 알림 시스템에 전송
}
}
}
```

문제 해결

일반적인 문제들

1. 피어 연결 실패

```
// 디버그 로깅 활성화
import "github.com/ipfs/go-log/v2"

logging.SetLogLevel("peering", "DEBUG")
```

```
// 또는 환경 변수로
// GOLOG_LOG_LEVEL=peering=DEBUG
```

2. 연결이 자주 끊어짐

```
// 연결 관리자 설정 확인
cm, err := connmgr.NewConnManager(
10, // 최소 연결 수 증가
200, // 최대 연결 수 증가
connmgr.WithGracePeriod(5*time.Minute), // 유예 기간 증가
)
```

3. 백오프 시간이 너무 김

```
// 현재 백오프 시간은 내부적으로 관리되며 수정할 수 없음
// 대신 여러 피어를 추가하여 가용성 향상
for _, backup := range backupPeers {
ps.AddPeer(backup)
}
```

네트워크 진단

```
func diagnosePeeringIssues(host host.Host, targetPeer peer.ID) {
```

```
// 연결 상태 확인
connectedness := host.Network().Connectedness(targetPeer)
log.Printf("연결 상태: %s", connectedness)
// 연결 시도
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
peerInfo := peer.AddrInfo{ID: targetPeer}
if err := host.Connect(ctx, peerInfo); err != nil {
log.Printf("연결 실패: %v", err)
} else {
log.Printf("연결 성공")
}
// 피어스토어 정보 확인
addrs := host.Peerstore().Addrs(targetPeer)
log.Printf("알려진 주소: %v", addrs)
}
```

이 문서는 Peering 모듈의 핵심 기능과 사용법을 다룹니다. 더 자세한 정보는 Bootstrap 부트스트랩과 Autoconf 자동 설정 문서를 참조하세요.